

Lab Assignment 5.1 and 6

2303a51227

Task 1:

Employee Data: Create Python code that defines a class named `Employee` with the following attributes: `empid`, `empname`, `designation`, `basic\_salary`, and `exp`. Implement a method `display\_details()` to print all employee details. Implement another method `calculate\_allowance()` to determine additional allowance based on experience:

If `exp > 10 years` → allowance = 20% of `basic\_salary`

If `5 ≤ exp ≤ 10 years` → allowance = 10% of `basic\_salary`

If `exp < 5 years` → allowance = 5% of `basic\_salary`

Finally, create at least one instance of the `Employee` class, call the `display\_details()` method, and print the calculated allowance.

```
5.1.1 lab.py > Employee > calculate_allowance
1  #Employee data
2  class Employee:
3      def __init__(self,empid,empname,designation,salary,experience=0):
4          self.empid=empid
5          self.empname=empname
6          self.designation=designation
7          self.salary=salary
8          self.experience=experience
9      def display_details(self):
10         print(f"Employee ID: {self.empid}")
11         print(f"Employee Name: {self.empname}")
12         print(f"Designation: {self.designation}")
13         print(f"Salary: {self.salary}")
14         print(f"Experience: {self.experience} years")
15     def calculate_allowance(self):
16         if self.experience>10:
17             allowance=0.20*self.salary
18         elif self.experience>=5 and self.experience<=10:
19             allowance=0.10*self.salary
20         else:
21             allowance=0.05*self.salary
22         print(f"Allowance: {allowance}")
23         print(f"Total Salary including Allowance: {self.salary + allowance}")
24 #Creating an Employee object
25 emp1=Employee(empid=101,empname="Naresh",designation="Software Engineer",salary=60000,experience=7)
26 emp1.display_details()
27 emp1.calculate_allowance()
```

Task 2:

Electricity Bill Calculation- Create Python code that defines a class named `ElectricityBill` with attributes: `customer\_id`, `name`, and `units\_consumed`. Implement a method `display\_details()` to print customer details, and a method `calculate\_bill()` where:

Units ≤ 100 → ₹5 per unit

101 to 300 units → ₹7 per unit More

than 300 units → ₹10 per unit

Create a bill object, display details, and print the total bill amount.

```
5.1.1 lab.py 5.1.2 lab.py X 5.1.3 lab.py 5.1.4 lab.py 5.1.5 lab.py 1 5.1.6 lab.py
5.1.2 lab.py > ...
1 #Electricity Bill Calculation
2 class Electricity_Bill:
3     def __init__(self, customer_id, customer_name, units_consumed):
4         self.customer_id = customer_id
5         self.customer_name = customer_name
6         self.units_consumed = units_consumed
7
8     def display_details(self):
9         print(f"Customer ID: {self.customer_id}")
10        print(f"Customer Name: {self.customer_name}")
11        print(f"Units Consumed: {self.units_consumed}")
12    def calculate_bill(self):
13        if self.units_consumed <= 100:
14            bill_amount = self.units_consumed * 5
15        elif 101 <= self.units_consumed <= 300:
16            bill_amount = (100 * 5) + (self.units_consumed - 100) * 7
17        else:
18            bill_amount = (100 * 5) + (200 * 7) + (self.units_consumed - 300) * 10
19        print(f"Total Bill Amount: {bill_amount}")
20 #Creating an Electricity_Bill object
21 bill1 = Electricity_Bill(customer_id=1035, customer_name="cvr", units_consumed=350)
22 bill1.display_details()
23 bill1.calculate_bill()
24 bill2 = Electricity_Bill(customer_id=1105, customer_name="harsh", units_consumed=250)
25 bill2.display_details()
26 bill2.calculate_bill()
```

Task 3:

Product Discount Calculation- Create Python code that defines a class named `Product` with attributes: `product\_id`, `product\_name`, `price`, and `category`. Implement a method `display\_details()` to print product details. Implement another method `calculate\_discount()` where:

Electronics → 10% discount

Clothing → 15% discount

Grocery → 5% discount

Create at least one product object, display details, and print the final price after discount.

```
5.1.1 lab.py 5.1.2 lab.py 5.1.3 lab.py X 5.1.4 lab.py 5.1.5 lab.py 1 5.1.6 lab.py single inheritance.p
5.1.3 lab.py > ...
1 #Product Discount Calculation
2 class Product_Discount:
3     def __init__(self, product_id, product_name, product_price ,category=None):
4         self.product_id = product_id
5         self.product_name = product_name
6         self.product_price = product_price
7         self.category = category
8
9     def display_details(self):
10        print(f"Product ID: {self.product_id}")
11        print(f"Product Name: {self.product_name}")
12        print(f"Product Price: {self.product_price}")
13        print(f"Category: {self.category}")
14
15    def calculate_discount(self):
16        if self.category == "Electronics" :
17            discount = self.product_price * 0.10
18        elif self.category == "Clothing":
19            discount = self.product_price * 0.15
20        elif self.category == "Groceries":
21            discount = self.product_price * 0.05
22        else:
23            discount = self.product_price * 0
24        final_price = self.product_price - discount
25        print(f"Discount Amount: {discount}")
26        print(f"Final Price after Discount: {final_price}")
27
28 #Creating Product_Discount objects
29 product1 = Product_Discount(product_id=201, product_name="Laptop", product_price=80000, category="Electronics")
30 product1.display_details()
31 product1.calculate_discount()
```

Task 4:

Book Late Fee Calculation- Create Python code that defines a class named `LibraryBook` with attributes: `book\_id`, `title`, `author`, `borrower`, and `days\_late`. Implement a method `display\_details()` to print book details, and a method `calculate\_late\_fee()` where:

Days late $\leq 5 \rightarrow ₹5$ per day

6 to 10 days late $\rightarrow ₹7$ per day

More than 10 days late $\rightarrow ₹10$ per day

Create a book object, display details, and print the late fee.

```
5.1 .1 lab.py 5.1.2 lab.py 5.1.3 lab.py 5.1.4 lab.py X 5.1.5 lab.py 1 5.1.6 lab.py single inheritance
5.1.4 lab.py > ...
1 #Book Late Fee Calculation
2 class Library_Book:
3     def __init__(self, book_id, book_title, author, borrower, days_late): ...
9
10     def display_details(self):
11         print(f"Book ID: {self.book_id}")
12         print(f"Book Title: {self.book_title}")
13         print(f"Author: {self.author}")
14         print(f"Borrower: {self.borrower}")
15         print(f"Days Late: {self.days_late}")
16
17     def calculate_late_fee(self):
18         if self.days_late <= 5:
19             late_fee = self.days_late * 5
20         elif 6 <= self.days_late <= 10:
21             late_fee = (5 * 5) + (self.days_late - 5) * 7
22         else:
23             late_fee = (5 * 5) + (5 * 7) + (self.days_late - 10) * 10
24         print(f"Total Late Fee: {late_fee}")
25
26 #Creating Library_Book objects
27 book1 = Library_Book(book_id=301, book_title="1984", author="George Orwell", borrower="lalith", days_late=1)
28 book1.display_details()
29 book1.calculate_late_fee()
30 book2 = Library_Book(book_id=302, book_title="To Kill a Mockingbird", author="Harper Lee", borrower="rohit", days_late=10)
31 book2.display_details()
32 book2.calculate_late_fee()
33 book3 = Library_Book(book_id=303, book_title="The Great Gatsby", author="F. Scott Fitzgerald", borrower="Cv", days_late=15)
34 book3.display_details()
35 book3.calculate_late_fee()
```

Task 5:

Student Performance Report - Define a function `student\_report(student\_data)` that accepts a dictionary containing student names and their marks. The function should:

Calculate the average score for each student

Determine pass/fail status (pass ≥ 40)

Return a summary report as a list of dictionaries

Use Copilot suggestions as you build the function and format the output.

```
5.1 .1 lab.py 5.1.2 lab.py 5.1.3 lab.py 5.1.4 lab.py 5.1.5 lab.py 1 X 5.1.6 lab.py single in
5.1.5 lab.py > ...
1 #Student Performance Report
2 def student_report(student_data):
3     student_data = {
4         "2303a51105": {"name": "Harsh", "math": 85, "science": 90, "english": 78},
5         "2303a51106": {"name": "Cvr", "math": 92, "science": 88, "english": 95},
6         "2303a51107": {"name": "Lalith", "math": 76, "science": 80, "english": 72}
7     }
8     average_scores = {}
9     for student_id, details in student_data.items():
10         total_score = details["math"] + details["science"] + details["english"]
11         average_score = total_score / 3
12         average_scores[student_id] = average_score
13         print(f"Student ID: {student_id}")
14         print(f"Name: {details['name']}")
15         print(f"Math: {details['math']}, Science: {details['science']}, English: {details['english']}")
16         print(f"Average Score: {average_score:.2f}\n")
17         def average_score(average_scores):
18             total = sum(average_scores.values())
19             count = len(average_scores)
20             overall_average = total / count
21             return overall_average
```

Task 6:

Taxi Fare Calculation-Create Python code that defines a class named `TaxiRide` with attributes: `ride\_id`, `driver\_name`, `distance\_km`, and `waiting\_time\_min`. Implement a method `display\_details()` to print ride details, and a method `calculate\_fare()` where:

- ₹15 per km for the first 10 km
- ₹12 per km for the next 20 km
- ₹10 per km above 30 km
- Waiting charge: ₹2 per minute

Create a ride object, display details, and print the total fare.

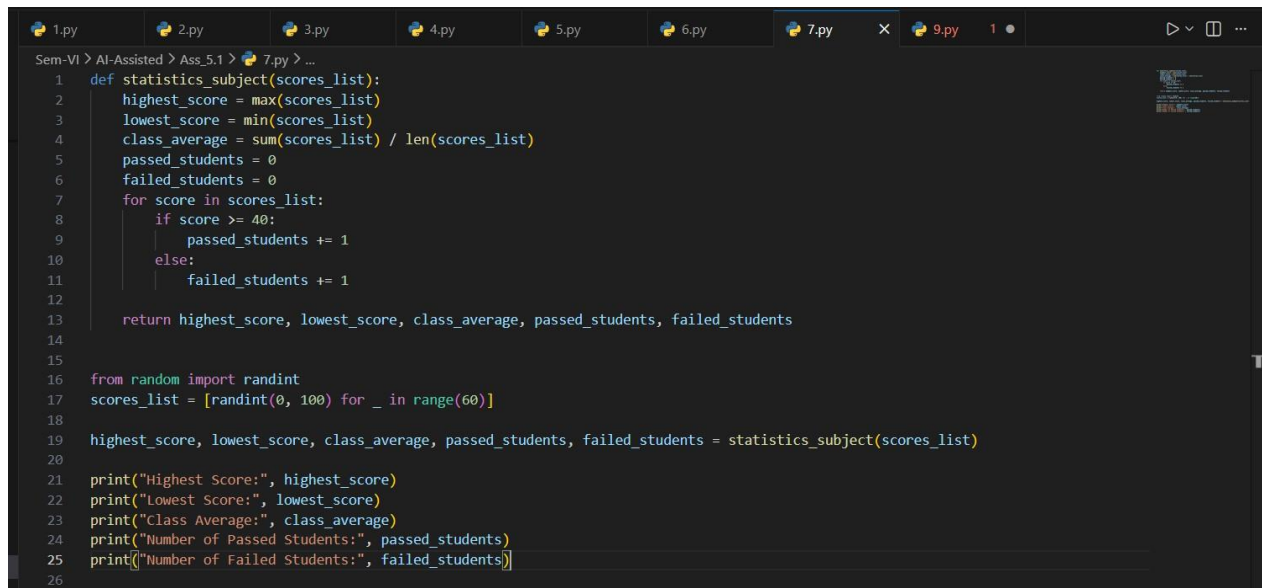
```
5.1.1 lab.py 5.1.2 lab.py 5.1.3 lab.py 5.1.4 lab.py 5.1.5 lab.py 1 5.1.6 lab.py X single inheritance
5.1.6 lab.py > ...
1  #Taxi Fare Calculation
2  class Taxi_Ride:
3      def __init__(self, ride_id, driver_name, distance_km, waiting_time_min=0):
4          self.ride_id = ride_id
5          self.driver_name = driver_name
6          self.distance_traveled_km = distance_km
7          self.waiting_time_min = waiting_time_min
8      def display_details(self):
9          print(f"Ride ID: {self.ride_id}")
10         print(f"Driver Name: {self.driver_name}")
11         print(f"Distance Traveled (km): {self.distance_traveled_km}")
12         print(f"Waiting Time (min): {self.waiting_time_min}")
13
14     def calculate_fare(self):
15         if self.distance_traveled_km <= 10:
16             fare = self.distance_traveled_km * 15
17         elif 10 < self.distance_traveled_km <= 30:
18             fare = (10*15) + (self.distance_traveled_km - 10)*12
19         elif self.distance_traveled_km > 30:
20             fare = (10*15) + (20*12) + (self.distance_traveled_km - 30)*10
21         else:
22             fare = 0
23         total_fare = fare + (self.waiting_time_min * 2)
24         print(f"Total Fare: {total_fare}")
25
26     #Creating Taxi_Ride objects
27     ride1 = Taxi_Ride(ride_id=401, driver_name="Ramesh", distance_km=25, waiting_time_min=10)
28     ride1.display_details()
29     ride1.calculate_fare()
```

Task 7:

Statistics Subject Performance - Create a Python function `statistics\_subject(scores\_list)` that accepts a list of 60 student scores and computes key performance statistics. The function should return the following:

- Highest score in the class
- Lowest score in the class
- Class average score
- Number of students passed (score ≥ 40)
- Number of students failed (score < 40)

Allow Copilot to assist with aggregations and logic



```
1 def statistics_subject(scores_list):
2     highest_score = max(scores_list)
3     lowest_score = min(scores_list)
4     class_average = sum(scores_list) / len(scores_list)
5     passed_students = 0
6     failed_students = 0
7     for score in scores_list:
8         if score >= 40:
9             passed_students += 1
10        else:
11            failed_students += 1
12
13    return highest_score, lowest_score, class_average, passed_students, failed_students
14
15
16 from random import randint
17 scores_list = [randint(0, 100) for _ in range(60)]
18
19 highest_score, lowest_score, class_average, passed_students, failed_students = statistics_subject(scores_list)
20
21 print("Highest Score:", highest_score)
22 print("Lowest Score:", lowest_score)
23 print("Class Average:", class_average)
24 print("Number of Passed Students:", passed_students)
25 print("Number of Failed Students:", failed_students)
26
```

Task 8: (Transparency in Algorithm Optimization)

Description:

Use AI to generate two solutions for checking prime numbers:

Naive approach(basic)

Optimized approach

Prompt:

“Generate Python code for two prime-checking methods and explain how the optimized version improves performance.”

Expected Output:

Code for both methods.

Transparent explanation of time complexity.

Comparison highlighting efficiency improvements.

```
Sem-VI > AI-Assisted > Ass.5.1 > 8.py > ...
1 """
2 Generate two Python code one is basic brute force whereas the other is optimized approach for two prime-checking methods
3 and explain how the optimized version improves performance.
4 """
5
6 def brute_force_prime(n):
7     # Handle edge cases:
8     # Numbers less than or equal to 1 are NOT prime
9     if n <= 1:
10         return False
11
12     # Try dividing n by every number from 2 to n-1
13     for i in range(2, n):
14         # If n is divisible by any number other than 1 and itself
15         if n % i == 0:
16             return False # Not a prime number
17
18     # If no divisors were found
19     return True
20
21 def optimized_prime(n):
22     # Handle edge cases first
23     if n <= 1:
24         return False
25
26     # 2 is the only even prime number
27     if n == 2:
28         return True
29
30     # Eliminate all other even numbers early
31     # This avoids unnecessary loop iterations
32     if n % 2 == 0:
33         return False
34
35     # Only check up to sqrt instead of n-1
36     # Because if n = a * b, at least one of them must be <= sqrt(n)
37     limit = int(n ** 0.5) + 1
38
39     # Skip even numbers by incrementing by 2
40     # This halves the number of iterations
41     for i in range(3, limit, 2):
42         if n % i == 0:
43             return False # Composite number found early
44
45     # If no divisors found up to sqrt
46     return True
47
48 # Example usage:
49 number = 29
50 print(f"Brute-force check: Is {number} prime? {brute_force_prime(number)}")
51 print(f"Optimized check: Is {number} prime? {optimized_prime(number)}")
```

Task 9: (Transparency in Recursive Algorithms)

Objective: Use AI to generate a recursive function to calculate Fibonacci numbers.

Instructions:

- Ask AI to add clear comments explaining recursion.
- Ask AI to explain base cases and recursive calls.

Expected Output:

- Well-commented recursive code.
- Clear explanation of how recursion works.
- Verification that explanation matches actual execution.

```
Sem-VI > AI-Assisted > Ass.5.1 > 9.py > ...
1 """Generate python code that develops fibonacci sequence using recursion Provide well commented and
2 docsting explanation for each step"""
3
4 def fibonacci_recursive(n):
5     """
6     Generate the nth Fibonacci number using recursion.
7
8     The Fibonacci sequence is defined as:
9     F(0) = 0
10    F(1) = 1
11    F(n) = F(n-1) + F(n-2) for n > 1
12
13    Parameters:
14    n (int): The position in the Fibonacci sequence to compute.
15
16    Returns:
17    int: The nth Fibonacci number.
18    """
19    # Base case: if n is 0, return 0
20    if n == 0:
21        return 0
22    # Base case: if n is 1, return 1
23    elif n == 1:
24        return 1
25    else:
26        # Recursive case: return the sum of the two preceding numbers
27        return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)
28
29 # Example usage:
30 position = 10
31 print(f"The {position}th Fibonacci number is: {fibonacci_recursive(position)}")
```


Task 10: (Transparency in Error Handling)

Task:

Use AI to generate a Python program that reads a file and processes data.

Prompt:

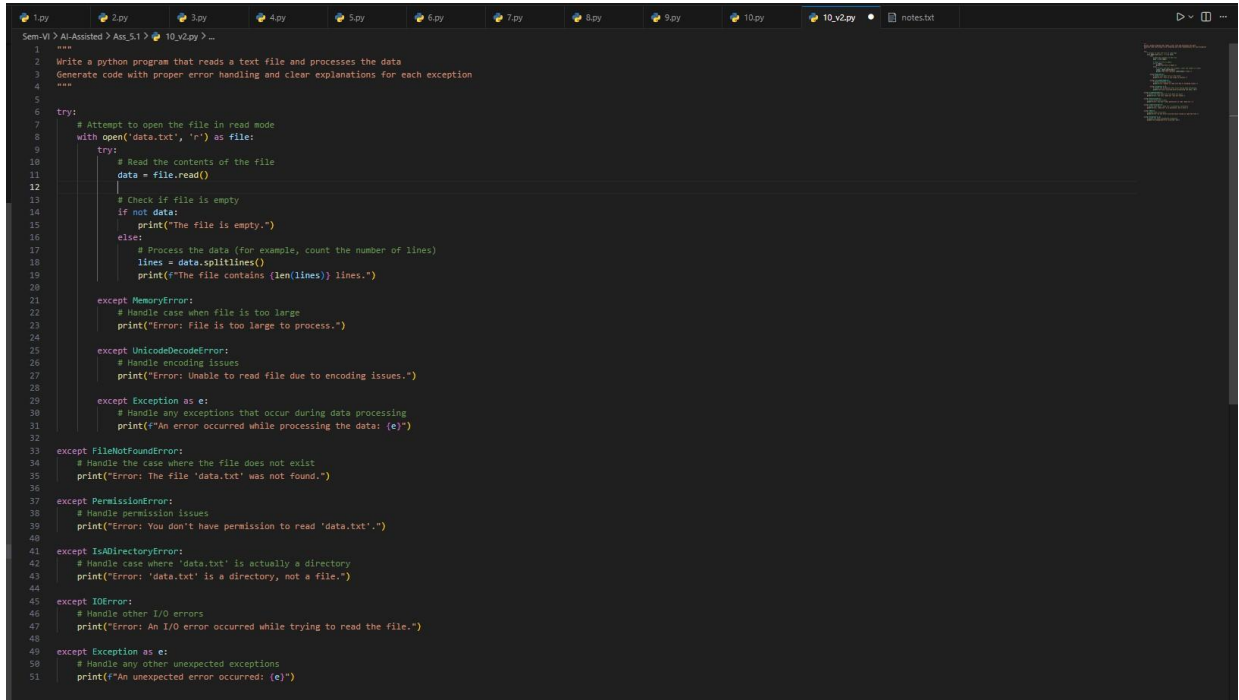
“Generate code with proper error handling and clear explanations for each exception.”

Expected Output:

Code with meaningful exception handling.

Clear comments explaining each error scenario.

Validation that explanations align with runtime behavior.



```
1  """
2  Write a python program that reads a text file and processes the data
3  Generate code with proper error handling and clear explanations for each exception
4  """
5
6  try:
7      # Attempt to open the file in read mode
8      with open('data.txt', 'r') as file:
9          try:
10             # Read the contents of the file
11             data = file.read()
12
13             # Check if file is empty
14             if not data:
15                 print("The file is empty.")
16             else:
17                 # Process the data (for example, count the number of lines)
18                 lines = data.splitlines()
19                 print(f"The file contains {len(lines)} lines.")
20
21         except MemoryError:
22             # Handle case when file is too large
23             print("Error: File is too large to process.")
24
25         except UnicodeDecodeError:
26             # Handle encoding issues
27             print("Error: Unable to read file due to encoding issues.")
28
29         except Exception as e:
30             # Handle any exceptions that occur during data processing
31             print(f"An error occurred while processing the data: {e}")
32
33     except FileNotFoundError:
34         # Handle the case where the file does not exist
35         print("Error: The file 'data.txt' was not found.")
36
37     except PermissionError:
38         # Handle permission issues
39         print("Error: You don't have permission to read 'data.txt'.")
40
41     except IsADirectoryError:
42         # Handle case where 'data.txt' is actually a directory
43         print("Error: 'data.txt' is a directory, not a file.")
44
45     except IOError:
46         # Handle other I/O errors
47         print("Error: An I/O error occurred while trying to read the file.")
48
49     except Exception as e:
50         # Handle any other unexpected exceptions
51         print(f"An unexpected error occurred: {e}")
```